EXPERIMENT 10 (PROJECT)

Name: Shivam Kumar
Uid: 23BCS14207
Section: KRG-1B

1. Aim

To design and analyze a **scalable real-time Airspace Defense Command Platform (Trinetra)** that allows operators to monitor airspace, deploy aircraft and drone swarms, detect incoming threats, predict enemy trajectories, and coordinate interception missions efficiently using **distributed system architecture, event streaming, and real-time communication technologies**.

---

2. Objective

- To identify **functional and non-functional requirements** of an airspace defense system
- To design **High Level Architecture (HLD)** for a large-scale command and control platform
- To understand **real-time telemetry streaming for aircraft and radar systems**
- To design **microservices for aircraft deployment, radar detection, mission planning, and combat simulation**
- To understand **AI-based threat prediction and interception mechanisms**
- To study **scalability challenges in large real-time simulation systems**

---

3. Tools

- **Draw.io / Excalidraw** – System architecture diagrams
- **PostgreSQL** – Persistent storage for missions, assets, and event logs
- **Redis** – Caching real-time aircraft and threat positions
- **Kafka** – Event streaming for telemetry, radar detections, and combat events
- **WebSockets** – Real-time updates for the tactical dashboard
- **Docker** – Containerized deployment of microservices
- **Load Balancer** – Distribute incoming traffic across services
- **Microservices Architecture** – Modular distributed services
- **Mapbox / Leaflet** – Tactical airspace visualization
- **AI Prediction Algorithms** – Threat trajectory prediction

---

4. System Requirements

A. Functional Requirements

• Operators should be able to **log in and access the command dashboard**
• System should display **real-time airspace map with aircraft, naval fleets, and threats**
• Operators should be able to **deploy aircraft, drone swarms, and naval fleets dynamically**
• System should detect **incoming enemy aircraft, drones, and missiles via radar simulation**
• System should continuously **track enemy movements and predict trajectories**
• Operators should be able to **assign interception missions to friendly aircraft**
• System should support **SAM Auto-Defense Zones** that automatically destroy threats entering their range
• Aircraft must **launch from the nearest airbase instead of spawning mid-air**
• System should provide **detailed metadata of each asset (speed, altitude, type, pilot info)**
• Operators should be able to **replay past events and analyze combat scenarios**
• System should generate **alerts for high-risk threats**

B. Non-Functional Requirements

1) Scalability

a. Support **500–1000 simultaneous aircraft and threats**
b. Handle **continuous telemetry updates from multiple sources**

2) Consistency

a. Mission assignments must be **accurate and conflict-free**
b. Prevent duplicate asset deployment or inconsistent mission states

3) Availability

a. Airspace monitoring dashboard must remain **always operational**
b. System must tolerate service failures without shutting down

4) Latency

a. Telemetry updates < **100 ms**
b. Radar detection events < **150 ms**

5) Reliability

a. No loss of simulation or mission data
b. Events must be durable through **Kafka event logs**

---

5. High Level Design (HLD)

The system follows **Microservices Architecture**.

Main Components

---

1) API Gateway

Responsibilities:

a. Authentication and authorization
b. Request routing to microservices
c. Rate limiting and traffic management
d. Centralized logging

---

2) Auth Service

Responsibilities:

a. User registration and login
b. Role-based access control
c. Token generation (JWT)

Database: PostgreSQL

---

3) Asset Management Service

Responsibilities:

a. Manage aircraft, drone swarms, naval fleets, and submarines
b. Track asset status and telemetry
c. Maintain asset metadata

---

4) Radar Detection Service

Responsibilities:

a. Simulate radar detection of enemy aircraft
b. Track incoming threats
c. Publish radar detection events to Kafka

---

5) Threat Analysis Service

Responsibilities:

a. Analyze enemy movement patterns
b. Calculate predicted trajectories
c. Assign threat priority levels

---

6) Simulation Engine

Responsibilities:

a. Continuously update positions of all aircraft and ships
b. Simulate movement physics and flight paths
c. Generate telemetry events

Telemetry events are sent to **Kafka topics**.

---

7) Mission Service

Responsibilities:

a. Assign fighter jets to intercept threats
b. Manage mission lifecycle
c. Update mission status

---

8) Combat Engine

Responsibilities:

a. Simulate missile launches and interceptions
b. Handle SAM defense zones
c. Determine combat outcomes

---

9) WebSocket Gateway

Responsibilities:

a. Push real-time updates to the frontend dashboard
b. Broadcast telemetry, mission updates, and combat events

Without WebSockets, the frontend would need to continuously poll the server.

With WebSockets:

- Aircraft positions update instantly
  - Radar detections appear immediately
  - Combat results are shown in real time

---

10) AI Prediction Service

Responsibilities:

a. Predict enemy trajectories
   b. Estimate interception points
   c. Rank threats based on danger level

---

6. Mission Execution Flow (Step-by-Step)

Operator logs into Trinetra command dashboard

System loads **real-time airspace map**

Radar Service detects incoming threat

Threat event published to **Kafka topic**

Threat Analysis Service predicts trajectory

Operator deploys fighter aircraft from nearest airbase

Mission Service assigns interception mission

Simulation Engine updates aircraft movement

Combat Engine simulates missile interception

WebSocket Gateway broadcasts results to UI

Dashboard updates with **interception outcome**

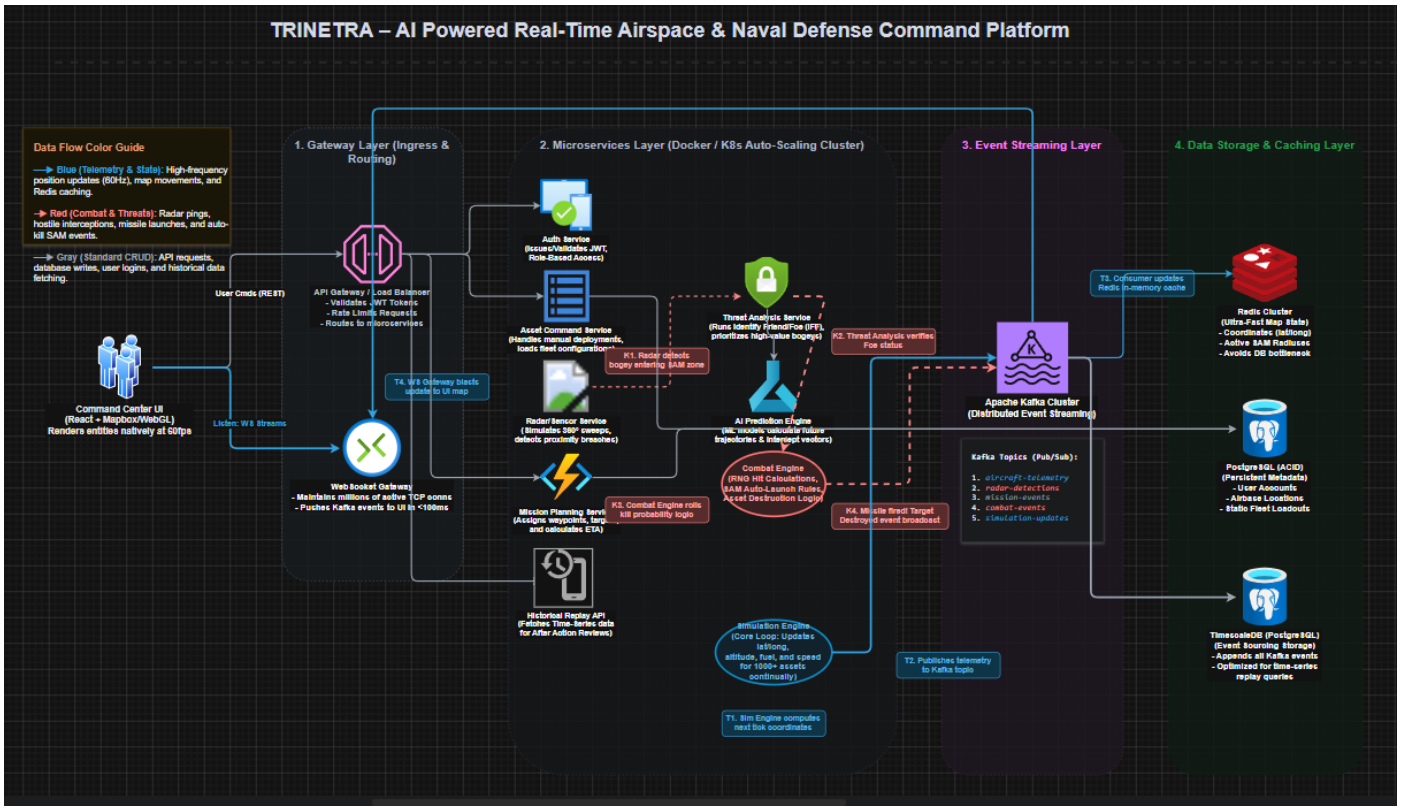
---

7. Scalability Solution

To support large-scale real-time simulations:

- Use **Kafka** for event streaming between services
  - Use **Redis** to cache aircraft and threat positions
  - Use **WebSockets** to push live updates to the UI
  - Use **Load Balancers** to distribute incoming traffic

  - Use **Microservices architecture** for modular scalability
  - Use **separate databases** for users, missions, and logs
  - Use **Docker containers** for scalable deployment
  - Use **airspace partitioning** to distribute simulation workload
  - Use **AI prediction services** to optimize mission decisions



---

8. Learning Outcomes (What I Have Learnt)

- Learned how **real-time command and control systems are designed**
  - Understood **event streaming architecture using Kafka**
  - Learned how **WebSockets enable real-time telemetry updates**
  - Understood **AI-based threat prediction mechanisms**
  - Learned how **microservices architecture supports scalability**
  - Understood how **radar detection and interception systems work**
  - Learned about **distributed simulation systems for large-scale environment**